



230968336

SAMEER RAJ

WEEK3

EXPERIMENT-1

```
In [1]: from collections import deque

def bfs(graph, start, goal):
    # Queue stores paths, not just nodes
    queue = deque()
    queue.append([start])

    # Visited set to avoid revisiting nodes
    visited = set()

    # Counter for expanded nodes
    nodes_expanded = 0

    while queue:
        # Remove the first path from the queue (FIFO)
        path = queue.popleft()
        current_node = path[-1]

        # Goal test
        if current_node == goal:
            return path, nodes_expanded

        # Expand node only if not visited
        if current_node not in visited:
            visited.add(current_node)
            nodes_expanded += 1

            # Generate successor states
            for neighbor in graph[current_node]:
                new_path = path + [neighbor]
                queue.append(new_path)

    # If goal is not reachable
    return None, nodes_expanded

maze = {
    'A': ['B', 'C'],
```

```

    'B': ['A', 'D', 'E'],
    'C': ['A', 'F'],
    'D': ['B'],
    'E': ['B', 'F'],
    'F': ['C', 'E', 'G'],
    'G': []
}

start_node = 'A'
goal_node = 'G'

path, expanded = bfs(maze, start_node, goal_node)

print("Path found:", path)
print("Nodes expanded:", expanded)

```

Path found: ['A', 'C', 'F', 'G']
Nodes expanded: 6

EXPERIMENT-2

```

In [2]: def dfs(graph, current, goal, path):
    # Add current node to the path
    path.append(current)

    # Check if goal is reached
    if current == goal:
        return path

    # Explore neighbors one by one (depth-first)
    for neighbor in graph[current]:
        # Avoid revisiting nodes already in the current path
        if neighbor not in path:
            result = dfs(graph, neighbor, goal, path)
            if result is not None:
                return result

    # Backtrack if goal not found in this path
    path.pop()
    return None

# Graph represented as an adjacency list
graph = {
    'A': ['B', 'C'],
    'B': ['A', 'D', 'E'],
    'C': ['A', 'F'],
    'D': ['B'],
    'E': ['B', 'F'],
    'F': ['C', 'E', 'G'],
    'G': []
}

```

```

# Initial and goal states
start_node = 'A'
goal_node = 'G'

# Run DFS
path = dfs(graph, start_node, goal_node, [])

# Display result
if path:
    print("Path found using DFS:", path)
else:
    print("No path found")

```

Path found using DFS: ['A', 'B', 'E', 'F', 'G']

EXPERIMENT-3

In [3]: **from** collections **import** deque

```

# Function to get valid moves of the blank tile
def get_neighbors(state):
    neighbors = []
    index = state.index(0)    # 0 represents the blank tile
    row = index // 3
    col = index % 3

    moves = {
        'UP': (-1, 0),
        'DOWN': (1, 0),
        'LEFT': (0, -1),
        'RIGHT': (0, 1)
    }

    for move in moves.values():
        new_row = row + move[0]
        new_col = col + move[1]

        if 0 <= new_row < 3 and 0 <= new_col < 3:
            new_index = new_row * 3 + new_col
            new_state = list(state)
            # Swap blank with adjacent tile
            new_state[index], new_state[new_index] = new_state[new_index], new_state[index]
            neighbors.append(tuple(new_state))

    return neighbors

# Breadth-First Search for 8-puzzle
def bfs(initial_state, goal_state):
    queue = deque()
    queue.append([initial_state])

```

```

visited = set()
visited.add(initial_state)

states_expanded = 0

while queue:
    path = queue.popleft()
    current_state = path[-1]
    states_expanded += 1

    # Goal test
    if current_state == goal_state:
        return path, states_expanded

    # Generate next states
    for neighbor in get_neighbors(current_state):
        if neighbor not in visited:
            visited.add(neighbor)
            queue.append(path + [neighbor])

    return None, states_expanded

# Function to print puzzle state nicely
def print_state(state):
    for i in range(0, 9, 3):
        print(state[i:i+3])
    print()

# Initial and Goal states
initial_state = (
    1, 2, 3,
    4, 0, 5,
    6, 7, 8
)

goal_state = (
    1, 2, 3,
    4, 5, 6,
    7, 8, 0
)

# Run BFS
solution_path, expanded = bfs(initial_state, goal_state)

# Display results
if solution_path:
    print("Solution Path:\n")
    for state in solution_path:
        print_state(state)

```

```
    print("Total states expanded:", expanded)
    print("Number of moves:", len(solution_path) - 1)
else:
    print("No solution found")
```

Solution Path:

(1, 2, 3)
(4, 0, 5)
(6, 7, 8)

(1, 2, 3)
(4, 5, 0)
(6, 7, 8)

(1, 2, 3)
(4, 5, 8)
(6, 7, 0)

(1, 2, 3)
(4, 5, 8)
(6, 0, 7)

(1, 2, 3)
(4, 5, 8)
(0, 6, 7)

(1, 2, 3)
(0, 5, 8)
(4, 6, 7)

(1, 2, 3)
(5, 0, 8)
(4, 6, 7)

(1, 2, 3)
(5, 6, 8)
(4, 0, 7)

(1, 2, 3)
(5, 6, 8)
(4, 7, 0)

(1, 2, 3)
(5, 6, 0)
(4, 7, 8)

(1, 2, 3)
(5, 0, 6)
(4, 7, 8)

(1, 2, 3)
(0, 5, 6)
(4, 7, 8)

(1, 2, 3)
(4, 5, 6)
(0, 7, 8)

(1, 2, 3)
(4, 5, 6)
(7, 0, 8)

(1, 2, 3)
(4, 5, 6)
(7, 8, 0)

Total states expanded: 6027
Number of moves: 14

EXPERIMENT-4

```
In [4]: from collections import deque
import time

# Graph representation
graph = {
    'A': ['B', 'C'],
    'B': ['A', 'D', 'E'],
    'C': ['A', 'F'],
    'D': ['B'],
    'E': ['B', 'F'],
    'F': ['C', 'E', 'G'],
    'G': []
}

# ----- BFS IMPLEMENTATION -----
def bfs(graph, start, goal):
    queue = deque([start])
    visited = set([start])

    nodes_expanded = 0
    max_queue_size = 1

    while queue:
        max_queue_size = max(max_queue_size, len(queue))
        path = queue.popleft()
        current = path[-1]
        nodes_expanded += 1

        if current == goal:
            return path, nodes_expanded, max_queue_size

        for neighbor in graph[current]:
            if neighbor not in visited:
                visited.add(neighbor)
                queue.append(path + [neighbor])

    return None, nodes_expanded, max_queue_size
```

```

# ----- DFS IMPLEMENTATION -----
def dfs(graph, current, goal, path, visited, depth_info):
    visited.add(current)
    path.append(current)

    depth_info["nodes_expanded"] += 1
    depth_info["max_depth"] = max(depth_info["max_depth"], len(path))

    if current == goal:
        return path

    for neighbor in graph[current]:
        if neighbor not in visited:
            result = dfs(graph, neighbor, goal, path, visited, depth_info)
            if result:
                return result

    path.pop()
    return None

# ----- PERFORMANCE COMPARISON -----
start_node = 'A'
goal_node = 'G'

# BFS Performance
start_time = time.time()
bfs_path, bfs_nodes, bfs_memory = bfs(graph, start_node, goal_node)
bfs_time = time.time() - start_time

# DFS Performance
dfs_info = {"nodes_expanded": 0, "max_depth": 0}
start_time = time.time()
dfs_path = dfs(graph, start_node, goal_node, [], set(), dfs_info)
dfs_time = time.time() - start_time

# ----- RESULTS -----
print("\nPERFORMANCE COMPARISON\n")

print("BFS Results:")
print("Path:", bfs_path)
print("Nodes Expanded:", bfs_nodes)
print("Execution Time:", bfs_time)
print("Maximum Memory Used (Queue Size):", bfs_memory)

print("\nDFS Results:")
print("Path:", dfs_path)
print("Nodes Expanded:", dfs_info["nodes_expanded"])
print("Execution Time:", dfs_time)
print("Maximum Memory Used (Recursion Depth):", dfs_info["max_depth"])

```

PERFORMANCE COMPARISON

BFS Results:

Path: ['A', 'C', 'F', 'G']

Nodes Expanded: 7

Execution Time: 3.7670135498046875e-05

Maximum Memory Used (Queue Size): 3

DFS Results:

Path: ['A', 'B', 'E', 'F', 'G']

Nodes Expanded: 7

Execution Time: 3.314018249511719e-05

Maximum Memory Used (Recursion Depth): 5

Post-Exercise Analysis Questions

1. Why does BFS guarantee an optimal solution while DFS does not?

Breadth-First Search guarantees an optimal solution because it explores the search space level by level. In an unweighted graph, all actions have the same cost, so the shallowest goal node corresponds to the shortest path. BFS always finds the goal at the minimum depth first, ensuring optimality. Depth-First Search, however, explores one branch completely before considering others and may reach the goal through a longer path even when a shorter path exists elsewhere, so it does not guarantee an optimal solution.

2. In which scenarios is DFS preferred over BFS?

Depth-First Search is preferred in scenarios where memory is limited because it stores only the current path rather than all nodes at a given level. It is also useful when the solution is expected to be deep in the search tree and when finding the shortest path is not a requirement. DFS is commonly used in problems involving backtracking, such as maze solving, puzzle solving, and constraint satisfaction problems.

3. How does the branching factor affect BFS performance?

The branching factor, which represents the average number of child nodes per state, has a significant impact on BFS performance. As the branching factor increases, the number of nodes generated at each level grows exponentially. This causes BFS to consume a large amount of time and memory because it must store and expand all nodes at each depth before moving to the next level.

4. Why is BFS considered impractical for large state spaces?

BFS is considered impractical for large state spaces because it requires a huge amount of memory to store all frontier nodes at each level of the search tree. Its time and space complexity grow exponentially with the depth of the solution and the branching factor. As a result, BFS becomes inefficient and difficult to scale for large or complex problems, even though it guarantees an optimal solution.

In []: